

Name: Monisha Raghini Chelle

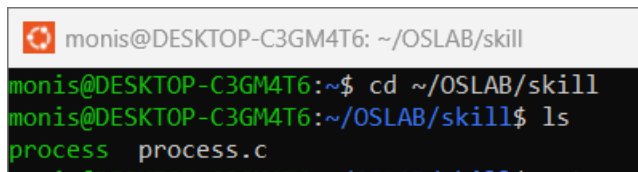
Roll No: 2520030537

OSSP (Operating Systems And Systems Programming) - 25CS2104E

(SKILL Content Delivery Concepts)

2. To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction

A terminal window screenshot with a title bar that reads 'monis@DESKTOP-C3GM4T6: ~/OSLAB/skill'. The terminal shows the following commands and output:

```
monis@DESKTOP-C3GM4T6:~$ cd ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls
process  process.c
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ cat process.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    char command[100];

    printf("Enter a Linux command: ");
    scanf("%99s", command);

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("Fork failed");
        return 1;
    }

    if (pid > 0)
    {
        // Parent Process
        printf("\nParent Process\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);

        wait(NULL);

        printf("\nChild process completed.\n");
    }
    else
    {
        // Child Process
        printf("\nChild Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        execlp(command, command, (char *)NULL);

        // Executes only if execlp() fails
        perror("Command execution failed");
        exit(1);
    }

    return 0;
}
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano shell.c
```

Paste this Code

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 shell.c *
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <termios.h>

#define BUFFER_SIZE 100

int main()
{
    struct termios oldt, newt;

    char buffer[BUFFER_SIZE];

    int length = 0;

    /* Get current terminal settings */
    tcgetattr(STDIN_FILENO, &oldt);

    /* Copy the settings */
    newt = oldt;

    /* Disable canonical mode and echo */
    newt.c_lflag &= ~(ICANON | ECHO);

    /* Apply new terminal settings */
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);

    printf("ossp> ");
    fflush(stdout);

    while (1)
    {
        char ch;

        /* Read one character at a time */
        read(STDIN_FILENO, &ch, 1);

        /* Enter key */
        if (ch == '\n' || ch == '\r')
        {
            buffer[length] = '\0';

            printf("\n");

            /* Exit condition */
        }
    }
}
```

Help **Write Out** **Where Is** **Cut** **Execute** **Location** **M-U** **Undo**
Exit **Read File** **Replace** **Paste** **Justify** **Go To Line** **M-E** **Redo**

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 shell.c *

    /* Exit condition */
    if (strcmp(buffer, "exit") == 0)
    {
        break;
    }

    printf("The command entered: %s\n", buffer);

    /* Clear input buffer */
    length = 0;

    printf("oss> ");
    fflush(stdout);
}

/* Backspace key */
else if (ch == 127 || ch == '\b')
{
    if (length > 0)
    {
        length--;

        printf("\b \b");
        fflush(stdout);
    }
}

/* Normal keyboard character */
else if (ch >= 32 && ch <= 126)
{
    if (length < BUFFER_SIZE - 1)
    {
        buffer[length] = ch;

        length++;

        printf("%c", ch);
        fflush(stdout);
    }
}

/* Restore original terminal settings */
tcsetattr(STDIN_FILENO, TCSANOW, &oldt);

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^I Paste      ^_ Justify    ^/ Go To Line  M-F Redo
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 shell.c *

    printf("The command entered: %s\n", buffer);

    /* Clear input buffer */
    length = 0;

    printf("ossp> ");
    fflush(stdout);
}

/* Backspace key */
else if (ch == 127 || ch == '\b')
{
    if (length > 0)
    {
        length--;

        printf("\b \b");
        fflush(stdout);
    }
}

/* Normal keyboard character */
else if (ch >= 32 && ch <= 126)
{
    if (length < BUFFER_SIZE - 1)
    {
        buffer[length] = ch;

        length++;

        printf("%c", ch);
        fflush(stdout);
    }
}

/* Restore original terminal settings */
tcsetattr(STDIN_FILENO, TCSANOW, &oldt);

printf("\nGoodbye!\n");

return 0;
}

Write to File: shell.c
^G Help      M-D DOS Format  M-A Append      M-B Backup File ^T Browse
^C Cancel    M-M Mac Format  M-P Prepend     ^O Discard buffer
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano shell.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls
process  process.c  shell.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc shell.c -o shell
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./shell
ossp>
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./shell
ossp> hello
The command entered: hello
ossp> hello world
The command entered: hello world
ossp> hello
The command entered: hello
ossp> hell
The command entered: hell
ossp> test
The command entered: test
ossp> exit

Goodbye!
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Your current folder is:

~/OSLAB/skill

and currently:

skill/

└─ process.c

Now we will add **Q2 separately**.

Q2 — Let's do it from your current folder

We will create:

skill/

└─ process.c ← Q1, already completed 

└─ shell.c ← Q2

1. Create Q2 file

```
cd ~/OSLAB/skill
```

```
nano shell.c
```

Purpose: Opens/creates the shell.c file for the Q2 shell program.

2. Save the Q2 code

Paste the given shell.c code and save:

Ctrl + O → Enter → Ctrl + X

3. Check files

```
ls
```

Expected:

```
process.c
```

```
shell.c
```

Purpose: Confirms that Q1 and Q2 files are present.

4. Compile Q2

```
gcc shell.c -o shell
```

Purpose: Compiles shell.c and creates the executable shell.

5. Run Q2

```
./shell
```

Expected:

```
ossp>
```

Purpose: Starts the interactive shell.

6. Test normal input

ossp> hello

The command entered: hello

Purpose: Tests keyboard input and the main loop.

7. Test multi-character input

ossp> hello world

The command entered: hello world

Purpose: Tests the input buffer and multi-character input.

8. Test Backspace

Type:

ossp> hello

Press **Backspace** once:

ossp> hell

Press Enter:

The command entered: hell

Purpose: Tests Backspace functionality.

9. Test Enter

Type:

ossp> test

Press **Enter**.

The command entered: test

Purpose: Tests Enter-key handling.

10. Test exit

ossp> exit

Goodbye!

Purpose: Tests the exit condition and terminates the shell.

Control Flow Diagram

CONTROL FLOW DIAGRAM

